

RETRIEVAL-AUGMENTED GENERATION (RAG)

Complete Guide · Basic to Advanced · Interview Ready

Chapter 1 — What is RAG & Why it Exists

The Problem RAG Solves

LLMs are trained on data up to a certain date. After that, they know nothing. They also cannot access your private company documents. And they sometimes make up facts — called hallucination.

RAG (Retrieval-Augmented Generation) solves all three of these problems by giving the LLM access to an external knowledge source at the moment it answers.

 **One-line definition:** RAG = Find relevant information first → then ask the LLM to answer using that information.

The Analogy

Think of two types of students taking an exam:

- **Without RAG:** Closed-book exam. Relies entirely on memorized knowledge. May be wrong or outdated.
- **With RAG:** Open-book exam. Can look up the answer in reference material first. More accurate, always up-to-date.

The LLM is the student. The vector database is the reference book. RAG turns a closed-book exam into an open-book exam.

What RAG Fixes

Problem	Without RAG	With RAG
Outdated knowledge	Only knows up to training cutoff date	Always retrieves fresh info from your docs
Hallucination on facts	Makes up specific facts it doesn't know	Answers grounded in real retrieved text
Private/company data	Never saw your internal documents	Reads your docs at query time
Source attribution	Cannot cite where it got information	Can point to exact source document + page
Scalability of knowledge	Adding knowledge = expensive retraining	Adding knowledge = just update the database

🎯 Interview tip: Always frame RAG around 3 problems: (1) stale knowledge, (2) hallucination on specific facts, (3) private data access. Interviewers love when you start from the problem, not the solution.

Chapter 2 — The Complete RAG Pipeline

RAG has two separate phases. Understanding this separation is fundamental.

Phase A — Indexing (Offline, done once)

This happens before any user asks a question. You prepare and store your knowledge base.

```
Your Documents (PDF, DOCX, TXT, HTML, database rows...)
↓
Step 1: LOAD      Read raw content from files
↓
Step 2: CLEAN     Remove headers/footers, fix encoding, strip HTML tags
↓
Step 3: CHUNK     Split into smaller pieces (200-500 words each)
↓
Step 4: EMBED     Convert each chunk to a vector using an embedding model
                  "The refund window is 30 days..." → [0.2, -0.4, 0.8, ...]
↓
Step 5: STORE     Save vectors + original text in a Vector Database
                  (each chunk stored with its vector + metadata)
↓
Vector Database is now ready to serve queries
```

💡 **Key point:** Indexing is a one-time cost. Once done, queries are fast. When documents change, you only re-index what changed — not everything.

Phase B — Retrieval + Generation (Online, every query)

This happens every time a user sends a message.

User asks: "What is the refund policy for digital products?"

↓

Step 1: EMBED QUERY Convert the question to a vector
"What is the refund policy..." → [0.1, -0.3, 0.9, ...]

↓

Step 2: SIMILARITY SEARCH Find the top-K chunks most similar to the query vector
(using cosine similarity across all stored vectors)
Returns: Chunk 3 (score 0.94), Chunk 17 (score 0.91)...

↓

Step 3: BUILD THE PROMPT Inject retrieved chunks into the prompt

```
System: You are a helpful assistant.  
       Answer ONLY from the provided context.  
  
Context:  
[Chunk 3]: "Digital products are non-refundable once  
           downloaded, except in cases of technical..."  
[Chunk 17]: "The standard refund window is 30 days..."  
  
User question: What is the refund policy for digital  
products?
```

↓

Step 4: LLM GENERATES Model reads context + question → produces grounded answer
Answer: "Digital products are non-refundable once downloaded..."

↓

(Optional) Step 5: CITE SOURCES Return which documents were used

🎯 Interview tip: Draw this pipeline when asked. Interviewers want to see you separate indexing (offline) from retrieval+generation (online). Many candidates confuse the two.

Chapter 3 — Chunking (The Hidden Bottleneck)

Chunking is how you split documents before embedding. It sounds simple but it has the biggest impact on RAG quality. Bad chunking = bad retrieval = bad answers, no matter how good your LLM is.

Why Chunking is Necessary

- **Embedding model limit:** Models like text-embedding-3-small can only process ~8,000 tokens at once. A 200-page PDF cannot be embedded as one piece.
- **Retrieval precision:** You want to retrieve the exact relevant paragraph, not an entire 50-page document. Small focused chunks = precise retrieval.
- **Context window limit:** Even if you could retrieve the whole document, you can't fit 200 pages into the LLM's context.

Chunking Strategies

1. Fixed-Size Chunking: Split every N characters or words, regardless of sentence or paragraph boundaries.

```
text = "The policy was updated in 2024. It now covers all..."
Chunk size = 50 chars:
  Chunk 1: "The policy was updated in 2024. It now c"
  Chunk 2: "overs all..." ← PROBLEM: word "covers" is split
```

- **Pros:** Simple, fast, consistent chunk sizes.
- **Cons:** Cuts sentences mid-thought, loses context at boundaries.
- **Use when:** Quick prototyping, uniform-structure data.

2. Sentence-Based Chunking: Split at sentence boundaries. Respect the natural flow of text.

```
Chunk 1: "The policy was updated in 2024."
Chunk 2: "It now covers all digital purchases made through our platform."
Chunk 3: "Refunds are processed within 5 business days."
```

- **Pros:** Clean boundaries, no broken sentences.
- **Cons:** Variable chunk sizes, short sentences may lack context.

3. Recursive Character Splitting (Recommended Default): Try to split at the most natural boundary first (paragraph → sentence → word → character), respecting a max chunk size.

```
Priority order:
1. Try to split at double newline (paragraph boundary) first
2. If chunk still too big → split at single newline
3. If still too big → split at sentence end (". ")
4. If still too big → split at space (word boundary)
5. Last resort → split at character
```

This is what LangChain's `RecursiveCharacterTextSplitter` does.

- **Pros:** Best balance of natural boundaries + consistent size.
- **Cons:** Still doesn't understand topic shifts.
- **Use:** Default for most RAG systems. Start here.

4. Semantic Chunking: Use an embedding model to detect when the topic shifts. Split when the semantic similarity between consecutive sentences drops significantly.

```
Compare embedding similarity between consecutive sentences:
Sentence 1 & 2: similarity = 0.92 → same topic, keep together
Sentence 2 & 3: similarity = 0.91 → same topic, keep together
Sentence 3 & 4: similarity = 0.45 → TOPIC CHANGE → split here
Sentence 4 & 5: similarity = 0.88 → same topic, keep together
```

- **Pros:** Chunks that actually represent one complete idea.

- **Cons:** Slow (requires embedding each sentence), harder to implement.
- **Use:** High-quality production systems where retrieval precision is critical.

5. Document-Structure Aware Chunking: Respect the inherent structure of your document type — headings, sections, tables, code blocks.

```
For a PDF with chapters:
  Chunk 1 = entire Section 1.1 (under heading "Refund Policy")
  Chunk 2 = entire Section 1.2 (under heading "Exchange Policy")

For HTML: split at <h2>, <h3> tags | For Markdown: split at ## headings
```

- **Pros:** Chunks have natural meaning — a whole section about one topic.
- **Cons:** Requires parsing the document structure, document-type-specific logic.

Chunk Size — The Key Tradeoff

Chunk Size	Retrieval Behavior	Best For
Small (100–200 words)	Very precise, but chunk may lack full context	FAQ, short facts, definitions
Medium (300–500 words)	Good balance — recommended default	Most use cases
Large (500–1000 words)	More context, but retrieval is less precise	Legal, research, technical docs

Overlap — Do Not Skip This

When chunking, the last N words of one chunk are repeated at the start of the next chunk. This prevents important information at the boundary from being missed.

```
Without overlap:
  Chunk 1: "...The customer must provide proof of purchase."
  Chunk 2: "This requirement applies to all returns..."
           ↑ 'This' has no context – the retriever doesn't know what it refers to

With overlap (50 words):
  Chunk 1: "...The customer must provide proof of purchase."
  Chunk 2: "provide proof of purchase. This requirement applies to all returns..."
           ↑ Now the context is clear
```

🎯 Interview answer on chunk size: Say 'I start with 300-500 words with 50-word overlap using recursive splitting. Then I evaluate retrieval quality and adjust based on my specific document type and query patterns.'

⚠ Common mistake: Forgetting overlap. Without it, facts that span chunk boundaries get lost. Always use 10-15% overlap relative to chunk size.

Chapter 4 — Embedding Models

The embedding model converts text into vectors. The quality of your embeddings directly determines retrieval quality. This is the second most important choice in RAG after chunking.

What an Embedding Does

```
Input: "The refund policy allows 30-day returns for physical goods."
Output: [0.21, -0.44, 0.87, 0.12, -0.33, 0.56, ...] (1536 numbers for OpenAI)
```

The same model also embeds the query:

```
Input: "Can I return a product after a month?"
Output: [0.19, -0.41, 0.89, 0.14, -0.31, 0.58, ...] ← very similar!
```

These two vectors are "close" in multi-dimensional space → retrieved correctly ✓

How Similarity is Measured — Cosine Similarity

Two vectors are similar if they point in the same direction in multi-dimensional space. Cosine similarity measures the angle between them.

$$\text{Cosine Similarity} = (A \cdot B) / (|A| \times |B|)$$

Score = 1.0 → identical direction (same meaning)

Score = 0.0 → perpendicular (unrelated)

Score = -1.0 → opposite direction (opposite meaning)

Practical ranges for RAG: > **0.85** (Very relevant), **0.70–0.85** (Somewhat relevant), < **0.70** (Probably not relevant).

Choosing an Embedding Model

Model	Provider	Dimensions	Max Tokens	Notes
text-embedding-3-small	OpenAI (API)	1536	8191	Fast, cheap, great quality. Best starting point.
text-embedding-3-large	OpenAI (API)	3072	8191	Better quality, 2× cost. Use for high-stakes retrieval.
bge-m3	BAAI (open)	1024	8192	Multilingual, free, runs locally. Very competitive.
e5-mistral-7b	Microsoft (open)	4096	4096	Top-tier quality, runs locally, needs GPU.
cohere-embed-v3	Cohere (API)	1024	512	Strong for English enterprise search.
all-MiniLM-L6-v2	HuggingFace	384	256	Very fast, small, good for prototyping. Limited context.

⚠ Critical rule: Always use the SAME embedding model for both indexing and querying. Different models produce incompatible vector spaces — the similarity scores would be completely meaningless.

Late Chunking vs. Early Chunking

- **Early chunking:** Split document into chunks, then embed each chunk independently. Each chunk's embedding has no context of the full document.
- **Late chunking (New):** Embed the FULL document first (or a long section), then split the embeddings. Each chunk's vector retains context from the surrounding text. Better retrieval for pronouns and broad references.

Early chunking: "Chunk: 'It was founded in 1995.' → embedding has no idea what 'It' refers to"
Late chunking: "Full doc embedded first → chunk knows 'It' = 'The company' from earlier text"

Chapter 5 — Vector Databases

A vector database stores your chunk embeddings and lets you search them by semantic similarity rather than exact keyword match. It is the core infrastructure of any RAG system.

What Makes a Vector DB Different

Feature	Traditional DB (SQL)	Vector Database
Query type	Exact match: WHERE name = 'Alice'	Similarity search: 'find chunks like this query'
Data stored	Rows with typed columns	Vectors (arrays of floats) + metadata
Index type	B-tree, Hash	ANN index (HNSW, IVF, LSH)
Result	Exact matches	Top-K nearest neighbors
Speed at scale	Fast for exact lookups	Fast for approximate similarity at scale

How Vector Search Works (ANN)

Finding the exact nearest vector across billions of records via brute force is too slow. Instead, vector DBs use **Approximate Nearest Neighbor (ANN)** algorithms. The most common is **HNSW (Hierarchical Navigable Small World)**, which builds a multi-layer graph where similar vectors are connected. Search starts at the top (coarse) layer and drills down to find precise neighbors quickly.

Layer 3 (sparse): ●—————● (coarse navigation)
 Layer 2 (medium): ●—●—————●—● (medium-grained search)
 Layer 1 (dense): ●●●●●●●●●●●●●●●● (fine-grained search)

Search: start at top layer, navigate to approximate region, drill down for precision.

💡 **Why ANN and not exact?** For 10M vectors, exact brute-force search requires 10M distance calculations per query. HNSW reduces this to $\sim \log(N)$ comparisons. The quality loss is tiny (usually 99%+ recall) while the speed gain is massive.

Popular Vector Databases Compared

Database	Type	Best For	Key Feature
FAISS	Open source, local	Research, fast prototyping	Fastest pure vector search, no persistence by default
ChromaDB	Open source, local	Small projects, local dev	Easiest to set up, 5 lines of code
Pinecone	Cloud (managed)	Production, teams	Fully managed, auto-scaling, no infra management
Weaviate	Open / Cloud	Hybrid search systems	Built-in BM25 + vector, GraphQL API
Qdrant	Open / Cloud	High performance + filtering	Fast payload filtering + vectors
pgvector	Postgres extension	Teams already on Postgres	Add vectors directly to your existing database
Milvus	Open / Cloud	Billion-scale deployments	Distributed architecture built for massive scale

🎯 Interview tip: Know at least 3 of these with their trade-offs. For a startup: ChromaDB (prototype) → Qdrant or Weaviate (production). For enterprise: Pinecone or Milvus. If already on Postgres: pgvector.

Metadata Filtering

Beyond vector similarity, you can filter results by metadata attached to each chunk. This is powerful for multi-tenant systems, language filtering, or domain-specific queries.

Each chunk is stored as:

```
{
  vector: [0.21, -0.44, 0.87, ...],
  text: "The refund window for digital products is...",
  metadata: { source: "policy_v3.pdf", department: "finance", date: "2024-01-15" }
}
```

Query with filter: `vector_search(query_vector, filter={ department: "finance", date: >=2024 })`

Chapter 6 — Retrieval Techniques

The retrieval step determines which chunks reach the LLM. Better retrieval = better answers.

1. Naive Vector Search (Baseline): Embed the query, run cosine similarity, and return the top-K chunks. Simple and fast, but fails on exact terms (product codes, names, abbreviations) and struggles with short queries.

2. Hybrid Search (Vector + BM25): Combine semantic vector search with traditional keyword search (BM25). The two scores are merged into a final ranking using Reciprocal Rank Fusion (RRF), catching cases where pure semantic search misses exact matches.

Feature	Vector Search	BM25 Keyword	Hybrid
Semantic understanding	✓ Strong	✗ Weak	✓ Strong
Exact keyword matching	✗ Weak	✓ Strong	✓ Strong
New/rare terms	✗ Misses them	✓ Finds them	✓ Finds them
Typo tolerance	✓ Handles well	✗ Fails	✓ Mostly OK

🎯 Interview tip: Hybrid search is the standard recommendation for production RAG. Always mention it as your default. Use $\alpha=0.5$ as a starting point, then tune based on query types.

3. Re-ranking: After initial fast-but-approximate retrieval, pass the top-K chunks through a re-ranker model (Cross-Encoder) that judges the query and context together for a high-precision relevance score.

- **Bi-encoder (Retrieval):** Embeds query and documents separately. Extremely fast; used to narrow down millions to top-50.
- **Cross-encoder (Re-ranking):** Processes query + document together. 100-1000× slower but highly accurate; run only on the top-50 to refine to top-5.

4. Maximal Marginal Relevance (MMR): Avoids returning redundant chunks that all state the same fact. MMR balances relevance with diversity, ensuring the retrieved context covers different aspects of the topic.

5. Parent-Child Retrieval (Small-to-Big): Index small, highly precise chunks for retrieval, but once matched, fetch and feed the larger parent section to the LLM to provide full context.

Chapter 7 — Generation & Prompt Design

The Standard RAG Prompt Template

```
SYSTEM: You are a helpful assistant for [Company Name].
Answer the user's question ONLY using the context provided below.
If the answer is not in the context, say "I don't have that information."
Do not make up information. Do not use knowledge outside the context.

CONTEXT:
[SOURCE 1 – refund_policy.pdf, Page 7]
"Digital products are non-refundable once downloaded, except in cases of technical failure..."

USER QUESTION: Can I get a refund on a digital product I downloaded yesterday?
ASSISTANT:
```

 **Key prompt rules:** 1) Direct the model to **ONLY** use the provided context. 2) Specify fallback phrases when text is insufficient. 3) Include source labels for citations. 4) Position context **BEFORE** the question to mitigate recency bias.

The Lost-in-the-Middle Problem

Research indicates that LLMs effectively utilize information at the absolute beginning and end of a context block, but often overlook or ignore details placed in the middle. To combat this:

- Place the highest scoring, most relevant chunks at the very top and bottom of the context layout.
- Limit total context intake to 3–5 highly relevant chunks rather than flooding the prompt.
- Always incorporate a high-quality re-ranking step.

Chapter 8 — Advanced RAG Techniques

1. Query Transformation: Rewriting or expanding user queries before running database retrieval.

- **Query Rewriting:** Clarifying vague text (e.g., "What about the policy?" → "refund and return policy terms and conditions").
- **Step-Back Prompting:** Generating a broader conceptual query to fetch foundational knowledge.
- **Multi-Query Generation:** Creating 3-5 structural variations of a query to maximize retrieval recall.
- **HyDE (Hypothetical Document Embeddings):** Asking the LLM to generate a fake placeholder answer, then embedding that fake answer to execute vector search. Answers align closer to documents in vector space than raw questions do.

2. Agentic RAG: Replacing static workflows with an agent loop (e.g., ReAct framework) that can recursively decide when to query, what database or tool to use, and how to aggregate multi-hop details.

3. Self-RAG & Corrective RAG (CRAG): Frameworks that evaluate retrieval quality programmatically. If similarity scores fall beneath a critical threshold, the pipeline automatically triggers fallback tasks like dynamic query rewriting or web search execution.

4. GraphRAG: Building a structured knowledge graph (entities and explicit relations) out of raw documents. Retrieval performs graph traversals, unlocking high-performance reasoning for complex relational or multi-hop enterprise queries.

Chapter 9 — Evaluation & Metrics

You must evaluate Retrieval performance and Generation performance separately to accurately diagnose failures.

Metrics Breakdown

- **Retrieval Metrics:** *Hit Rate (Recall@K)* (was the right chunk found?), *MRR (Mean Reciprocal Rank)* (how high was the right chunk ranked?), and *Context Precision & Recall*.
- **Generation Metrics:** *Faithfulness* (is the answer grounded in the context without hallucination?), *Answer Relevancy* (does it address the prompt?), and *Answer Correctness* (factual alignment with reference truth).

🎯 Interview tip: When asked how you evaluate RAG, cite the **RAG Triad** (TruLens framework): Context Relevance, Groundedness, and Answer Relevance. Mention measuring them via **RAGAS** with an LLM judge.

Chapter 10 — RAG vs Fine-Tuning

Dimension	RAG	Fine-Tuning
What it changes	What the model KNOWS per query	How the model BEHAVES always
Knowledge freshness	Real-time — update DB anytime	Static — requires retraining to update
Private data	Never bakes data into model weights	Data baked into weights permanently
Source attribution	Natural — explicit chunk citations	Hard — model simply 'knows' things
Hallucination	Lower — heavily grounded in text	Higher — relies on lossy memory compression
Best for	Factual Q&A, Dynamic knowledge bases	Style, tone, formatting, unique behavior

Best Practice: Use both together. Fine-tune for custom structural behavior (brand voice, specialized syntax), and run RAG to feed precise, up-to-date factual metrics.

Chapter 11 — Interview Questions & Answers

Q: What is chunking? What chunking strategy would you recommend?

Chunking is splitting documents into smaller pieces before embedding because embedding models have token limits and smaller chunks yield precise retrieval. I recommend starting with **recursive character splitting** (300-500 words with 10-15% overlap). For production, evaluate **semantic chunking** for ideal conceptual coherence, or structure-aware chunking to protect core document hierarchies. Overlap is crucial to avoid losing facts sliced by strict boundaries.

Q: What is hybrid search? Why is it better than pure vector search?

Hybrid search combines semantic vector search with keyword-based search (BM25), merging results via Reciprocal Rank Fusion (RRF). Pure vector search fails on explicit alphanumeric terms, serial IDs, or product codes. Hybrid search offers the best of both worlds and acts as the production default for robust RAG applications.

Q: How would you debug a RAG system that gives wrong answers?

Isolate the pipeline into two distinct checkpoints. First, test **Retrieval**: inspect the top-K chunks manually. If the right information isn't there, fix chunking, add hybrid search, or apply re-ranking. Second, test **Generation**: inject the correct source chunk manually into the prompt. If the LLM still fails, improve the system prompt instructions or switch to a more capable model.

Q: What are the primary failure modes of RAG?

1) *Retrieval Miss*: information exists but isn't fetched (solve with hybrid search/re-ranking). 2) *Context Ignoring*: LLM answers using internal weights instead of prompt details (strengthen prompt constraints). 3) *Fragmented Context*: chunk lacks core surrounding references like pronouns (solve with parent-child retrieval or overlap). 4) *Version Conflict*: older document revisions clash with new records (solve using strict metadata date filters).

— End of RAG Study Guide —